

Lab 07

SE3082 – Parallel Computing

IT24610790

Exercise 7: Summary Table & Thinking Answers

1. Collective Comparison Table

Program	Collectives Used	Memory Allocation	Root Needs Manual Sum Loop?	Where Final Result is Available
Exercise 1 (sum_bcast)	MPI_Bcast	Full array on all processes	Yes (via MPI_Recv loop)	Root only
Exercise 2 (sum_scatter)	MPI_Scatter	Full array on root, chunk on workers	Yes (via MPI_Recv loop)	Root only
Exercise 3 (sum_gather)	MPI_Scatter, MPI_Gather	Full array on root, chunk on workers	Yes (sums gathered array)	Root only
Exercise 4 (sum_reduce)	MPI_Scatter, MPI_Reduce	Full array on root, chunk on workers	No (handled internally by MPI)	Root only
Exercise 5 (sum_allreduce)	MPI_Scatter, MPI_Allreduce	Full array on root, chunk on workers	No (handled internally by MPI)	All processes
Exercise 6 (sum_scan)	MPI_Scatter, MPI_Scan	Full array on root, chunk on workers	No (prefix reduction handled by MPI)	Different per rank (cumulative)

2. Execution Time & Performance Note

- **Network and Memory Efficiency:** Using MPI_Scatter minimizes memory footprint and network overhead by distributing only the relevant partition of the array to each process, rather than replicating the entire array via broadcast.
- **Tree-based Communication:** Collective operations such as MPI_Reduce and MPI_Allreduce utilize internal tree-based algorithms operating in $O(\log P)$ steps, executing significantly faster than a sequential $O(P)$ loop of point-to-point MPI_Send and MPI_Recv operations.

3. Thinking Question Answer

While MPI_Allreduce distributes the identical global aggregate to all processes, MPI_Scan computes an inclusive prefix reduction where each process receives the cumulative result corresponding to ranks 0 through its own rank.

MPI_Scan is preferred in scenarios requiring running totals or distinct boundary offsets across distributed ranks, such as:

1. **Parallel Prefix Sums / Cumulative Distribution:** Computing cumulative frequency or empirical distribution functions across partitions.
2. **Global Offset Calculation for Parallel I/O:** Determining file write positions where each process writes a variable number of records sequentially into a shared file without overlapping.
3. **Dynamic Work Balancing and Indexing:** Assigning unique, monotonically increasing global indices across distributed work items without inter-process coordination.